Task B - Lessons Learned and Issues Encountered

What I Learned

The biggest lesson from this assignment was how much Cloudflare Zero Trust depends on policy composition rather than a single product toggle. Access, Gateway, WARP, identity providers, tunnels, and SaaS integrations all solve different parts of the problem. The final design works because those controls are layered together.

I also learned that Cloudflare Access can provide a consistent policy layer across very different application types. A self-hosted application behind Cloudflare Tunnel and a SaaS application using SAML can both be governed by the same group and allowlist model. That is a useful pattern because the operator does not need a separate access-control strategy for every application.

Another important lesson was how Cloudflare Gateway evaluates policy order. The Entertainment filtering requirement looked simple at first, but the exception for Netflix and YouTube changes the design. The allow rule for Netflix and YouTube has to be placed before the broader Entertainment block rule. Otherwise the broad category block wins first and the exception never applies.

Issues Encountered

Google Group Claims Were Not Available

The assignment references user groups, but the identity provider available in this environment is a personal Gmail account. Personal Gmail does not expose Google Workspace group claims to Cloudflare Access. That means I could not rely on directory-synced Google groups for Testers, SaaS-Users, or Admins.

I handled this by using Cloudflare-native Access groups and reusable policies backed by explicit email includes. This preserves the policy shape required by the assignment while documenting the limitation clearly. In a production Google Workspace tenant, I would replace those email includes with real IdP group claims.

Gateway Rule Ordering Was Easy to Get Wrong

The HTTP filtering requirement asked for Entertainment to be blocked except Netflix and YouTube. The first attempt can easily be implemented as a single category block, but that blocks the exceptions too.

I resolved this by treating the exception as the first policy:

1. Allow Netflix and YouTube domains.
2. Block the broader Entertainment category.

I also documented the ordering requirement at https://github.com/mareox/Cloudflare-Project/blob/main/docs/zero-trust/gateway-policies/README.md so the next operator understands why the allow rule must stay above the block rule.

API Writes Were Not Reliable From This Environment

The local Cloudflare API connection could read account, zone, tunnel, DNS, Access, Workers,

and Zero Trust state, but write attempts returned an authentication error. Where direct API writes were not available, the implementation was captured through dashboard configuration and exported as reusable JSON policy definitions.

The result is still reviewable and repeatable because the important control decisions are stored in the public repo under:

- https://github.com/mareox/Cloudflare-Project/tree/main/docs/zero-trust/access-policies
- https://github.com/mareox/Cloudflare-Project/tree/main/docs/zero-trust/gateway-policies

SaaS SSO Requires a Real SaaS Admin Surface

Cloudflare can act as the identity provider for SaaS applications, but the SaaS side still needs an administrator to paste the SAML metadata or OIDC client details. That makes SaaS SSO harder to demonstrate than a self-hosted Access app, especially in a personal lab environment.

I handled this by documenting the Cloudflare-side SaaS application shape and the expected SAML/OIDC configuration flow at https://github.com/mareox/Cloudflare-Project/blob/main/docs/zero-trust/access-policies/saas-app.json.

How I Overcame the Issues

I kept the design aligned to the intent of the assignment instead of pretending the personal lab environment had enterprise-only identity features. The main compromises were documented directly in the deliverables:

- Use Cloudflare-native Access groups when IdP group claims are unavailable.
- Preserve default-deny behavior even when using email includes.
- Store policies as JSON so they can be recreated or moved to a different account.
- Document ordering-sensitive Gateway rules near the policy files.
- Keep screenshots and policy files together so the implementation can be validated from the repo.

The end result is a practical Zero Trust deployment model that can run in a small lab but still maps cleanly to a production implementation.